

Documentation Technique — Pipeline CI/CD

Projet : CrisisView — Tableau de bord de gestion d'incidents

État du projet

Frontend Fonctionnel

- Pipeline CI/CD complet : Checkout → Install → Test → SonarQube → Build → Docker → Deploy
- 6 tests automatisés (unitaires, intégration, snapshot)
- Analyse SonarCloud fonctionnelle
- Image Docker publiée sur Docker Hub
- Déploiement des fichiers sur la VM Azure effectif
- **Limitation** : configuration Nginx manquante pour servir l'application sur le port 80

Backend Fonctionnel

- Pipeline CI/CD complet : Checkout → Install → Test → SonarQube → Deploy
- 6 tests unitaires réussis (calculateDistance)
- Analyse SonarCloud fonctionnelle
- Déploiement des fichiers sur la VM Azure effectif
- **Limitation** : le test d'intégration nécessite une base MySQL non disponible dans Jenkins. L'image Docker du backend n'est pas construite dans le pipeline.
- **Solutions** : ajout d'un service MySQL dans le pipeline Jenkins ; ajout d'un stage Docker Build.

Difficultés techniques rencontrées

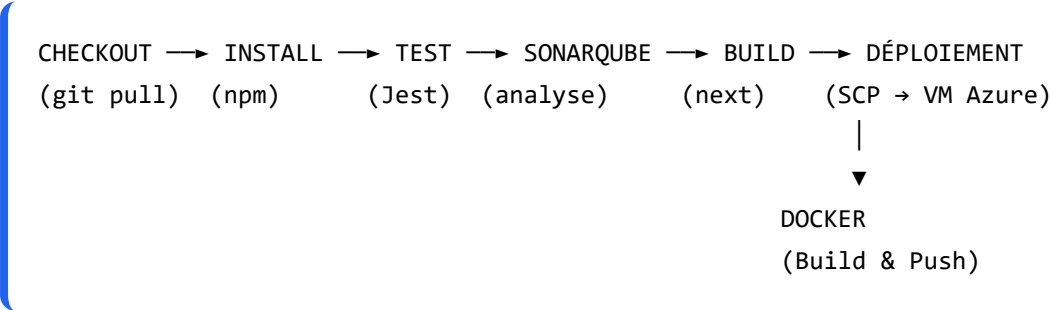
- **Incompatibilité Next.js 9 / App Router** : le projet initial utilisait Next.js 9 (Pages Router) avec une structure App Router (Next.js 13+). Migration complète vers Next.js 14 nécessaire.
- **Conflits de dépendances** : incompatibilités entre React 19, Next.js 9 et react-

leaflet. Résolution via migration des versions dans package.json.

- **Build Docker** : erreurs OpenSSL avec Node.js 18, résolues par l'utilisation de Node.js 16 et NODE_OPTIONS.
- **Tests d'intégration backend** : dépendance à MySQL non satisfaite dans Jenkins. Contournement : exécution des tests unitaires uniquement.
- **Permissions Azure** : ajustement des droits sur les dossiers de déploiement via sudo chown.
- **SonarQube Backend** : erreur "file indexed twice" résolue en excluant les doublons dans sonar-project.properties.

Partie 1 : Architecture de la pipeline CI/CD

1.1 Schéma global



1.2 Enchaînement des étapes

Étape	Commande	Rôle
Checkout	checkout scm	Récupère le code source depuis GitHub
Install	npm install	Installe les dépendances Node.js
Test	npm test -- --coverage	Exécute 6 tests Jest (unitaires, intégration, snapshot)
SonarQube	sonar-scanner	Analyse la qualité du code
Build	npm run build	Compile l'application Next.js pour la production
Docker	docker build && docker push	Construit l'image et la publie sur Docker Hub
Deploy	scp	Déploie les fichiers sur la VM Azure

1.3 Outils utilisés

Outil	Rôle
Jenkins (Docker local)	Orchestrateur CI/CD

GitHub	Hébergement du code source
Jest + React Testing Library	Tests automatisés
SonarCloud	Analyse statique du code (SAST)
Next.js 14	Framework frontend React
Docker + Docker Hub	Conteneurisation et registre d'images
Azure VM (Ubuntu 24.04)	Serveur de déploiement

| 1.4 Types de tests

Type	Nombre	Description
Unitaires	3 (frontend) + 6 (backend)	Validation de fonctions pures
Intégration	2 (frontend)	Rendu des composants React
Snapshot	1 (frontend)	Détection de changements structurels

Partie 2 : Runbooks

Runbook 1 : Exécuter le projet en local

Prérequis

- Node.js 18 ou supérieur
- Git
- Accès au dépôt GitHub

Étape 1 : Cloner le dépôt

```
git clone https://github.com/[utilisateur]/CrisisView-Pipeline.git
cd CrisisView-Pipeline
```

Étape 2 : Lancer le frontend

```
cd frontend
npm install --legacy-peer-deps
npm run dev
```

Accessible sur : <http://localhost:3000>

Étape 3 : Lancer le backend

```
cd backend
npm install
npm start
```

Accessible sur : <http://localhost:3001>

Runbook 2 : Déployer via Docker Compose

Prérequis

- Docker et Docker Compose installés

Étape 1 : Lancer les conteneurs

```
cd CrisisView-Pipeline
docker-compose up -d
```

Étape 2 : Vérifier

```
docker ps
```

- crisisview-frontend sur le port **3000**
- crisisview-backend sur le port **3001**

Runbook 3 : Relancer un pipeline Jenkins

Prérequis

- Jenkins démarré sur `http://localhost:8080`
- Jobs `CrisisView-Pipeline` et `CrisisView-Backend` configurés

Étape 1 : Accéder à Jenkins

- URL : `http://localhost:8080`

Étape 2 : Lancer le build

1. Cliquer sur le job souhaité
2. **Build Now** (menu de gauche)
3. Surveiller dans **Build History**
4. Console Output pour voir les logs

Étape 3 : Résultats

- Dashboard SonarCloud Frontend : [https://sonarcloud.io/dashboard?id=\[organisation\]_CrisisView-Pipeline](https://sonarcloud.io/dashboard?id=[organisation]_CrisisView-Pipeline)
- Dashboard SonarCloud Backend : <https://sonarcloud.io/dashboard?id=CrisisView-Backend>
- Image Docker Hub : [https://hub.docker.com/r/\[utilisateur\]/crisisview-frontend](https://hub.docker.com/r/[utilisateur]/crisisview-frontend)

Annexes

A. Configuration Jenkins (Credentials)

Credential ID	Type	Usage
azure-vm-ip	Secret text	Adresse IP de la VM Azure
azure-vm-user	Secret text	Nom d'utilisateur SSH
azure-ssh-key	SSH Private Key	Clé privée de connexion
sonarcloud-token	Secret text	Token d'authentification SonarCloud
docker-hub-pwd	Secret text	Token d'accès Docker Hub

B. Résultat des tests



```
=== Frontend ===
Test Suites : 3 passed, 3 total
Tests       : 6 passed, 6 total
Snapshots   : 1 passed, 1 total
Couverture  : 42.3%

=== Backend ===
Test Suites : 1 passed, 1 total
Tests       : 6 passed, 6 total
Couverture  : 50%
```

C. Résultat SonarCloud



```
=== Frontend ===
Quality Gate : Not computed
Open Issues  : 0
Duplications : 0.0%
Coverage     : 9.1%

=== Backend ===
```


Analyse : ANALYSIS SUCCESSFUL
